Bootstrapped Distillation for Cheaper Pre-training

Luka Ivanić*[1], Denis Ibiši[1], and Teo Verbanac[1]

[1]M.Sc. Student in Software Engineering, Faculty of Engineering, University of Rijeka, Croatia

January 6, 2026

Abstract

Training large language models (LLMs) is increasingly compute-intensive, motivating methods that reduce the compute required to reach a target *validation loss* during pre-training. We study knowledge distillation as a compute-saving technique and propose a *bootstrapped distillation* procedure: train a smaller teacher model first, then distill a larger student using logits-based KL divergence while the teacher outperforms the student, and finish training classically. On Wikitext-103, this reduces the compute required to reach the target validation loss by 8% for a 70M student and 20% for a 150M student, both distilled from a 17M teacher; importantly, the reported compute includes the FLOPs required to train the teacher. Supporting ablations show that most distillation gains occur very early and that learning-rate schedules dominate late-stage progress.

1 INTRODUCTION

Progress in the performance of LLMs is strongly linked to scale, as seen in models such as GPT-1 [1], BERT [2], Megatron-LM [3], and GPT-3 [4]. Scaling-law studies show that pre-training loss decreases predictably with model size, data, and training compute [5, 6]. However, the compute required to reach increasingly lower loss targets is growing rapidly, motivating practical methods that reduce training cost.

Knowledge distillation (KD) transfers information from a trained teacher model to a student model, often accelerating convergence and improving training efficiency [7, 8, 9]. In this work, we focus on KD as a compute-saving strategy for reaching a *target validation loss* during pre-training. Our key idea is a *bootstrapped distillation* procedure: train a smaller teacher first, train the larger student with logits distillation while the teacher outperforms the student, stop distillation once the student is within a small margin of the teacher (we use $\sim 5\%$ validation-loss), and finish the remaining training classically. This can be interpreted as using the teacher to skip the slow early optimization steps on the large model.

We make the following contributions:

- We define a practical bootstrapped KD procedure with a simple stopping rule for transitioning back to classical training.
- On Wikitext-103, we demonstrate 8% compute savings for a 70M student and 20% savings for a 150M student, both distilled from a 17M teacher.
- We provide ablation evidence that KD gains concentrate in the early phase of training and that the learning-rate schedule becomes the dominant bottleneck later, informing how KD should be scheduled.

The paper is structured as follows. Section 2 describes the dataset, model architecture, and training paradigms. Section 3 defines our evaluation and compute accounting. Section 4 presents the main results and supporting ablations, followed by discussion and limitations in Section 5 and concluding remarks in Section 6.

*Main contributor

2 METHODOLOGY

This section outlines the complete methodology used in our research, from data acquisition and preprocessing to model architecture, the knowledge distillation framework, and the training and inference procedures.

2.1 Dataset and Preprocessing

Our work is based on the Wikitext-103-v1 dataset, used in prior LLM studies such as GPT-1 [1] and BERT [2]. The dataset is a large, high-quality corpus of English text derived from verified "Good" and "Featured" articles on Wikipedia, making it a suitable and challenging benchmark for causal language modeling. We prepare the dataset through a two-stage process: we first train a Byte-Pair Encoding (BPE) tokenizer and then implement a data pipeline to transform the raw text into fixed-length training examples.

- **Tokenizer Generation:** We train a custom BPE tokenizer from scratch on the training split of the dataset. The tokenizer employs a ByteLevel pre-tokenizer for robustness to any input text and applies NFKC Unicode normalization. It is configured with a vocabulary size of $5,000$ (we justify this choice in Section 3.1) and contains special tokens for unknown words ([UNK]), padding ([PAD]), Beginning-of-Sequence ([CLS]) and End-of-Sequence ([SEP]).
- **Data Pipeline:** For causal language model training, the raw text articles undergo a sequential transformation. Each article is first tokenized, with a [SEP] token appended to mark document boundaries. All tokenized articles are then concatenated into a single, continuous stream of token IDs. To preserve document boundaries from the raw Wikitext-103 dataset, our pipeline uses regular expressions to identify section titles (e.g., `= Section Title =`) as delimiters, ensuring that semantically distinct articles are not arbitrarily merged before being segmented. The stream is subsequently segmented into fixed-length blocks matching the model's context length. For the causal modeling task, the input sequence for each block is used directly, while the target labels are created by shifting the same sequence one position to the right. Any final, shorter sequence is discarded from the dataset.

2.2 Model Architecture

We design and implement a decoder-only Transformer model from the ground up. The architecture is defined by a stack of identical Transformer blocks, preceded by embedding layers and followed by a final language model head.

- **Embeddings:** The model uses two embedding layers: a standard token embedding layer and a learnable positional embedding layer, as introduced in the original Transformer architecture [10]. Their outputs are summed to provide the input to the first Transformer block.
- **Transformer Block:** Each block consists of two main sub-layers:

  1. A multi-head self-attention (MHA) mechanism that computes a weighted sum over all input values. The weights are determined by the similarity between a *query* from the current token and the *keys* of all other tokens. This process is performed in parallel across multiple 'heads' to capture diverse contextual relationships. An optimized implementation leverages FlashAttention [11] for computational efficiency. Finally, dropout is applied to the attention scores.
  2. A position-wise feed-forward network (FFN), consisting of two linear layers with a GELU activation function, as used in BERT [2] and Megatron-LM [3]. The hidden dimension of the FFN is four times the model's primary dimension. A dropout layer is applied to its output.

  The architecture follows a Pre-LN configuration: both sub-layers are preceded by a normalization layer and followed by a residual connection.
- **LM Head:** The final output layer projects the transformer's final hidden state back into the vocabulary space to generate token logits. This layer effectively functions as a "de-embedding" layer, reversing the initial token embedding process. Its weights are tied with the token embedding matrix, following

strategies used in models like BERT [2] and T5 [12], to reduce the total number of parameters and improve performance.

- **Weight Initialization:** We employ a specific weight initialization scheme to enhance training stability. All weights are initialized from a normal distribution with a mean of 0 and a standard deviation of $\texttt{d\_model}^{-0.5}$, where d_{model} represents the dimensionality of the model's vector representations. Furthermore, we apply a scaled residual initialization strategy. The output projection weights in the multi-head self-attention (MHA) and the second linear layer in the FFN are scaled by a factor of $1/\sqrt{2 \cdot N_{\text{layers}}}$, with N_{layers} being the total number of Transformer blocks. This ensures that at the start of training, the contribution of each Transformer block is minimal, forcing the information to flow primarily through the residual pathway. This encourages the model to learn transformations gradually, preventing the variance of activations from growing and maintaining stability in deep models.

2.3 Training paradigms

Our research evaluates training paradigms that aim to reduce the compute required to reach a target validation loss during pre-training. We implement a classical baseline, a standard teacher-student knowledge distillation framework, and a bootstrapped distillation procedure where a smaller model is trained first and then used to accelerate the early training of a larger model.

- **Classical Training:** The baseline for our experiments is established through a classical training procedure informed by the principles of compute-optimal training and scaling laws outlined in seminal works by Kaplan et al. [5] and Hoffmann et al. [6]. Models are trained from scratch to minimize the cross-entropy loss between their predictions and the target tokens. Critically, rather than strictly adhering to a fixed recipe, we conduct **extensive hyperparameter sweeps** to determine the final, optimized hyperparameters for each model size, as presented in Table 1. This empirical approach ensures that each baseline model is robust and serves as a fair benchmark for comparison.

- **Knowledge Distillation (Teacher-Student):** This paradigm transfers knowledge from a pre-trained teacher model to a student model. The student learns to match the teacher's output probability distribution by minimizing the Kullback-Leibler (KL) Divergence between their softened logits (temperature $\tau = 2$). The student minimizes a combined objective consisting of the standard cross-entropy loss on ground-truth labels and a dynamically normalized distillation loss:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{CE}} + \mathcal{L}'_{\text{distill}} \tag{1}$$

  To keep the distillation signal strong as the student approaches the teacher, we normalize the distillation loss in each step so that its batch mean matches the batch mean of $\mathcal{L}_{\text{CE}}$. The normalized distillation loss, $\mathcal{L}'_{\text{distill}}$, is computed as in equation (2), where $\overline{\mathcal{L}}_{\text{CE}}$ and $\overline{\mathcal{L}}_{\text{distill}}$ are batch means.

$$\mathcal{L}'_{\text{distill}} = \mathcal{L}_{\text{distill}} \times \frac{\overline{\mathcal{L}}_{\text{CE}}}{\overline{\mathcal{L}}_{\text{distill}}} \tag{2}$$

- **Bootstrapped Distillation (Small→Large):** We use distillation to accelerate the early training of a larger model using a smaller teacher. We first train a smaller teacher model classically. We then train the larger student with logits-based KL distillation using the same dynamically normalized distillation loss from equation (2) while the teacher outperforms the student. Distillation is stopped once the student's validation loss is within a small margin of the teacher's (we use $\sim 5\%$), after which training continues classically. This procedure is designed to skip slow early optimization steps of the larger model.

2.4 Training Procedure

The specific hyperparameters for each model are centrally configured and detailed in Table 1. The main components of our training procedure include:

Table 1: Key hyperparameters for classical model training across sizes. Model names (e.g., 70M) refer to approximate non-embedding parameters; the #Params column shows total parameters including embeddings.

Model	#Params	Emb.	Layers	Heads	Ctx.	Batch (tok)	$\mathbf{LR_{max}}$	$\mathbf{LR_{min}}$
1M Classic	1.7M	128	5	8	512	16k	$1.0e-3$	$1.0e-4$
2M Classic	2.89M	128	11	8	512	16k	$1.0e-3$	$1.0e-4$
5M Classic	6.28M	256	6	8	1024	16k	$1.0e-3$	$1.0e-4$
10M Classic	14.9M	256	13	8	1024	16k	$8.0e-4$	$8.0e-5$
17M Classic	18.45M	384	8	16	1024	16k	$1.0e-3$	$1.0e-4$
30M Classic	34.59M	512	10	16	1024	32k	$8.0e-4$	$8.0e-5$
70M Classic	75.56M	512	23	16	1024	32k	$6.0e-4$	$6.0e-5$
150M Classic	154.33M	512	48	16	1024	32k	$3.0e-4$	$3.0e-5$

Table 2: Wikitext-103-v1 Dataset Statistics

Statistic	Training	Validation	Test
Number of Articles	28,475	60	60
Number of Lines	2.3M	4.9k	5.8k
Number of Words	101.4M	213.9k	241.2k
Number of Characters	539.5M	1.1M	1.3M
Avg. Words per Article	3,625	3,627	4,092

- **Optimizer and Scheduler:** We use AdamW with weight decay of 0.1. The learning rate uses linear warmup over the first 2,000 steps followed by cosine decay over 10,800 total steps. Learning-rate sensitivity notes are provided in Appendix B.
- **Precision and Hardware:** Training is performed on NVIDIA A100 40GB GPUs. We leverage Automatic Mixed Precision (AMP) to accelerate training, with support for both `bfloat16` and `float16` data types. When using `float16`, a gradient scaler is employed to prevent underflow issues with small gradients.
- **Regularization:** We use dropout with rate 0.1 and gradient clipping by norm (max norm 1.0), as done in Megatron-LM [3] and also applied in GPT-1 [1] and GPT-3 training [4], to prevent exploding gradients during training.
- **Experiment Tracking:** Reproducibility and detailed analysis are ensured through integration with Weights & Biases. Our pipeline automatically logs all hyperparameters, training and validation metrics, and model checkpoints.

3 CASE STUDY

This section details the experimental setup for our case study, including the dataset composition, data partitioning strategy, and the metrics used to evaluate the performance and efficiency of our distillation-based compute-saving procedures.

3.1 Dataset

To provide a clear understanding of the dataset's scale and composition, its raw statistics are presented in Table 2.
Selecting a vocabulary size involves a trade-off between token-to-word efficiency and embedding parameter inflation. A larger vocabulary improves compression (fewer tokens per word), but the vocabulary size

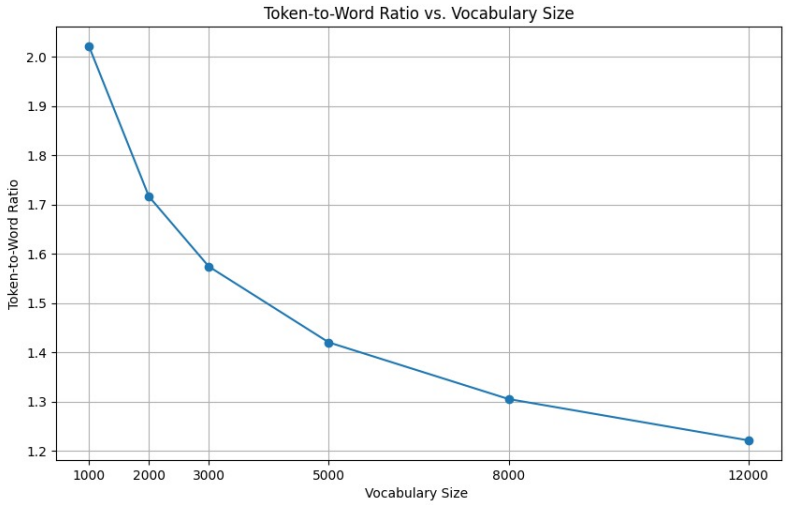

Figure 1: Token-to-word ratio vs vocabulary size on Wikitext-103. Compression efficiency improves rapidly up to ∼5k vocabulary, with diminishing returns beyond.

directly dictates the number of parameters in the token embedding and language model head layers, as given by equation (3), where V is the vocabulary size and d is the hidden dimension.

$$\text{Number of embedding parameters} = V \cdot d \tag{3}$$

For the tiny models in this study (< 5M parameters), a vocabulary exceeding 10k would inflate embedding parameters to dominate the total parameter budget, leaving insufficient capacity for the Transformer blocks. Fig. 1 shows the token-to-word ratio as a function of vocabulary size on Wikitext-103. The curve demonstrates that 5,000 is a soft inflection point: compression improves rapidly up to this size, with diminishing returns beyond it.
We therefore selected a vocabulary size of 5,000, balancing reasonable compression against a lean parameter budget appropriate for small-scale language models.

3.2 Context Length Selection

Preliminary experiments show that small models can be penalized by overly long contexts, while larger models benefit from longer context windows. Based on this analysis, we use a context length of 512 tokens for models below 3M parameters and 1024 tokens for larger models. The full context-length sweep is provided in Appendix A (figures 4 and 5).

3.3 Evaluation

We evaluate language modeling capability and training efficiency using the metrics described below.

3.3.1 Performance Metrics

- **Cross-Entropy Loss:** The primary metric is the average cross-entropy loss $\mathcal{L}_{\text{CE}}$ over the validation set, measuring how well the model predicts the next token.
- **Positional Loss Analysis:** As a prerequisite analysis for establishing strong classical baselines on smaller models and context sizes, we examine cross-entropy loss across different positions in the input sequence. The context window is divided into segments of 64 tokens, and the average loss is calculated for each segment. This allows us to observe whether predictive accuracy degrades as the context grows longer (see Appendix A).

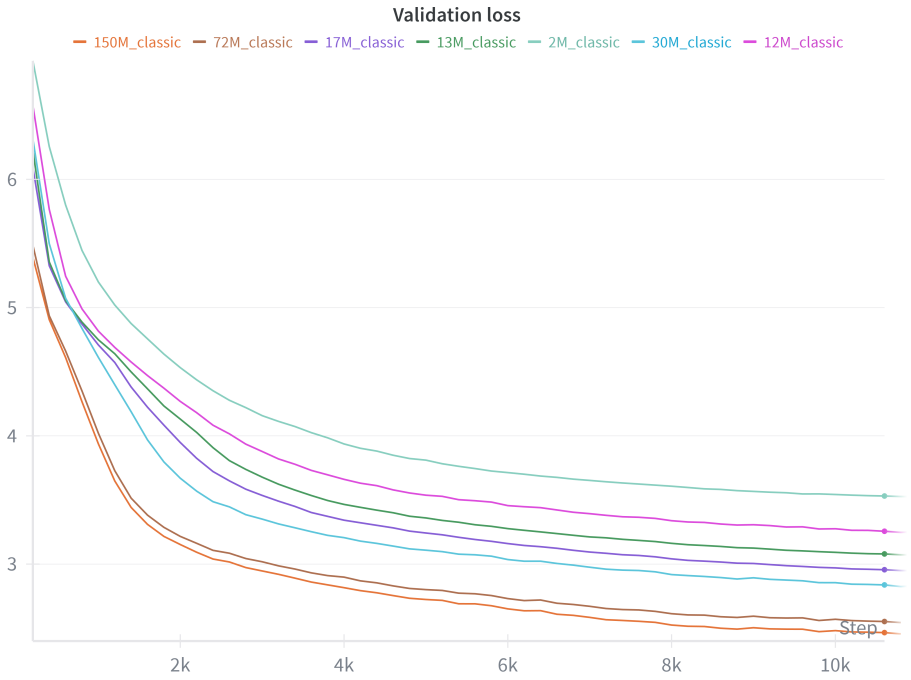

Figure 2: Validation loss curves for classically trained models of varying sizes

3.3.2 Efficiency Metrics

A key goal of this research is to reduce training compute. We report:

- **Model Size:** The total number of trainable parameters N.
- **Compute Proxy:** We estimate training compute using a Kaplan-style approximation [5]:

$$C \approx 6NT \tag{4}$$

  where N is the number of model parameters and T is the total number of training tokens processed (batch size in tokens $\times$ training steps).

Compute savings are reported as the relative reduction in total compute required to reach the target validation loss compared to a classical baseline for the same student model. For bootstrapped distillation, the total compute includes (i) the compute required to train the teacher model, (ii) the inference compute of the teacher during distillation, and (iii) the compute required to train the student until it reaches the target. In this paper, the target is defined as the validation loss achieved by the classical baseline at the end of its scheduled training run (10.8k steps).

4 RESULTS

This section presents the empirical results of our study. We first show representative classical baselines, then present the main result: bootstrapped logits distillation reduces the training compute required to reach a target validation loss.

4.1 Classical Baselines

We trained a suite of classical baselines across model sizes. Larger models achieve lower loss for the same training length, consistent with scaling behavior (Fig. 2).

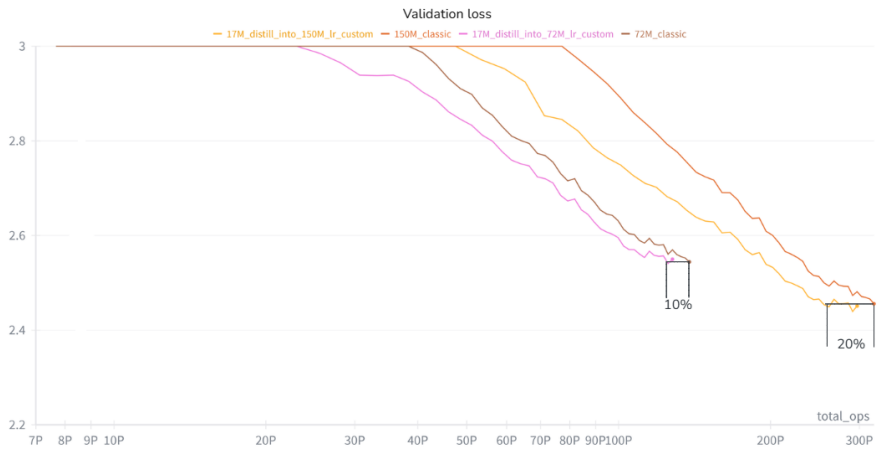

Figure 3: Validation loss vs compute (in PFLOPs, i.e. 10^{15} floating-point operations) showing 8.0% compute savings for 17M→70M and 20.3% savings for 17M→150M. Totals include teacher-training compute, teacher-inference compute during distillation, and student-training compute.

Configuration	Target Loss	Steps	Compute (PFLOPs)	Savings
70M Classical	2.54	10,800	138	—
70M Bootstrapped	2.54	10,000	127	8.0%
150M Classical	2.45	10,800	320	—
150M Bootstrapped	2.45	10,000	255	20.3%

Table 3: Bootstrapped distillation results. Compute savings are relative to classical training for the same student model. For 150M Bootstrapped, we report the compute when the target loss was first reached.

4.2 Bootstrapped Distillation Reduces Compute to Target Loss

We evaluate the bootstrapped distillation procedure described in Section 2: train a smaller teacher, distill a larger student with logits KL using the dynamically normalized distillation loss in equation (2) while the teacher outperforms the student, stop distillation once the student is within $\sim 5\%$ validation loss of the teacher, and finish training classically. Compute is estimated using $C \approx 6NT$ (Section 3); importantly, for bootstrapped runs we report total compute including the teacher's training compute. The target validation loss is defined as the loss achieved by the classical baseline at the end of its scheduled run (10.8k steps).
Fig. 3 shows validation loss vs compute for both cases, and Table 3 summarizes the numerical results. We observe 8.0% compute savings for 17M→70M and 20.3% compute savings for 17M→150M (including teacher-training compute). Compute is reported in PFLOPs (10^{15} floating-point operations).

5 DISCUSSION

Our results position knowledge distillation as a practical tool for reducing the compute required to reach a target validation loss during pre-training. The key contribution is *bootstrapped distillation*: using a smaller teacher to accelerate the early training of a larger student and switching to classical training once the student approaches the teacher.

5.1 Why Bootstrapped Distillation Works

The teacher provides a strong early training signal that helps the student reach a useful optimization region faster than classical training alone, after which classical training can refine performance beyond the teacher.

This approach is particularly relevant when no pretrained model exists for the target dataset or domain. In industry settings, pretrained LLMs are often readily available as teachers; however, when training on novel data, bootstrapped distillation provides a practical way to create a teacher cheaply and still benefit from KD speedups.

Distillation accelerates the early phase of training: in our experiments, the first ∼2160 steps (20% of total training) under classical training are condensed into ∼1080 steps when distilling (10% of total). Distillation continues until approximately step 1800, after which classical training resumes. The later distillation steps (from ∼1080 to ∼1800) are less impactful—we believe this is because further loss improvements require the learning rate to be lower, allowing the model to refine its state more precisely. A higher learning rate during the initial distillation phase (steps 0–1080) enables rapid progress, but eventually the model reaches a "loss plateau" in terms of what the current learning rate can achieve.

We find a strong link between learning rate and the minimal loss achievable under classical training (Appendix B, figure 7). It is therefore vital to lower the learning rate as quickly as possible to enable faster convergence—but not so quickly that early training is slowed. This presents a difficult optimization problem that KD-aware schedules can help address (figure 8).

5.2 Limitations

Compute is approximated via $C \approx 6NT$ rather than exact hardware-level FLOPs. Results are limited to a single dataset (Wikitext-103) and model family up to 150M parameters, and we do not exhaustively sweep learning-rate schedules or random seeds.

6 CONCLUSION

This work investigated knowledge distillation as a compute-saving technique for reaching a target validation loss during pre-training. We introduced a bootstrapped distillation procedure: train a smaller teacher first, distill a larger student with logits KL while the teacher outperforms the student, stop distillation once the student approaches the teacher, and finish training classically.

Using $C \approx 6NT$ as a compute proxy, we observed 8% compute savings for a 70M student and 20% savings for a 150M student when bootstrapped from a 17M teacher. Ablations support a simple interpretation: distillation provides most of its value early, while late-stage progress is dominated by learning-rate scheduling. Future work should validate these findings at larger scales and on additional datasets, refine KD-aware learning-rate schedules, and strengthen compute accounting with more explicit FLOPs measurements and multiple random seeds.

References

[1] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, "Improving Language Understanding by Generative Pre-Training," OpenAI, Tech. Rep. GPT-1, 2018.

[2] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," *arXiv preprint arXiv:1810.04805*, 2018.

[3] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, "Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism," *arXiv preprint arXiv:1909.08053*, 2019.

[4] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language Models are Few-Shot Learners," *arXiv preprint arXiv:2005.14165*, 2020.

[5] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, "Scaling Laws for Neural Language Models," *arXiv preprint arXiv:2001.08361*, 2020.

[6] J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. d. L. Casas, L. A. Hendricks, J. Welbl, A. Clark, T. Hennigan, E. Noland, K. Milican, G. v. d. Driessche, B. Damoc, A. Guy, S. Osindero, K. Simonyan, E. Elsen, J. W. Rae, O. Vinyals, and L. Sifre, "Training Compute-Optimal Large Language Models," *arXiv preprint arXiv:2203.15556*, 2022.

[7] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," *arXiv preprint arXiv:1910.01108*, 2019.

[8] X. Jiao, Y. Yin, L. Shang, X. Jiang, X. Chen, L. Li, F. Wang, and Q. Liu, "TinyBERT: Distilling BERT for Natural Language Understanding," *arXiv preprint arXiv:1909.10351*, 2019.

[9] Y. Gu, L. Dong, F. Wei, and M. Huang, "MiniLLM: Knowledge Distillation of Large Language Models," *arXiv preprint arXiv:2306.08543*, 2024.

[10] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention Is All You Need," *arXiv preprint arXiv:1706.03762*, 2017.

[11] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and Memory-Efficient Exact attention with IO-Awareness," *arXiv preprint arXiv:2205.14135*, 2022.

[12] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer," *arXiv preprint arXiv:1910.10683*, 2020.

A Context Length Impact Analysis

Context length is a key training design choice that interacts strongly with model size. Small models can be penalized by overly long contexts (optimization difficulty and wasted capacity), while larger models benefit from longer contexts. For the experiments in this paper, we therefore use a context length of 512 tokens for models below 3M parameters and 1024 tokens for larger models, as described in Section 3. This appendix provides the supporting sweeps.

In the plots below, each curve corresponds to a separate model trained with a given *maximum* context length (shown in the legend). Each point reports the average validation loss over tokens whose positions fall into a fixed position bucket; the x-axis indicates the bucket's upper bound. Models contribute points only up to their training maximum (e.g., a model trained with a 512-token context has points at 64/128/256/512).

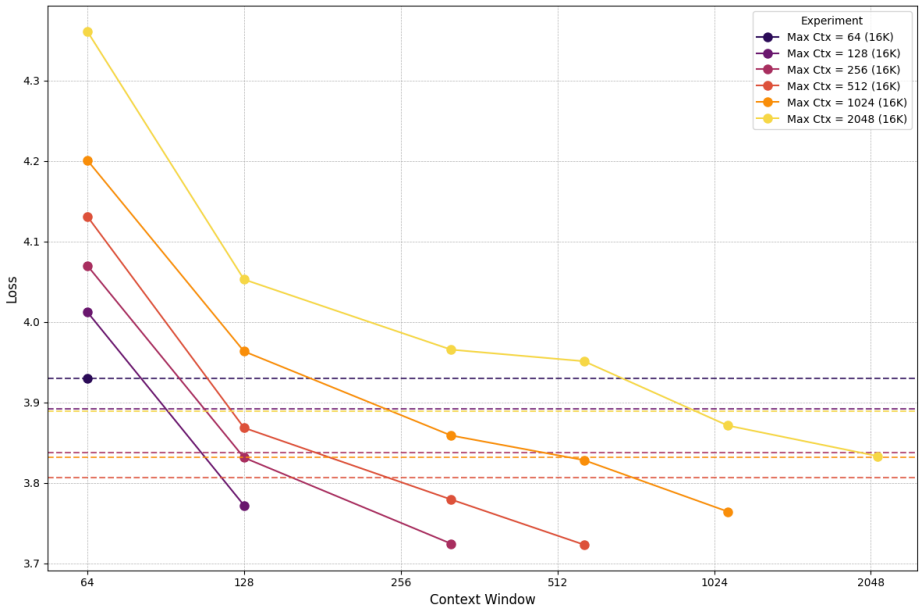

Figure 4: Context-length sweep for the 1M-parameter model (batch size 16k tokens). The 1M model does not handle maximum context lengths above 512 well: as the training maximum increases beyond 512, overall validation loss degrades.

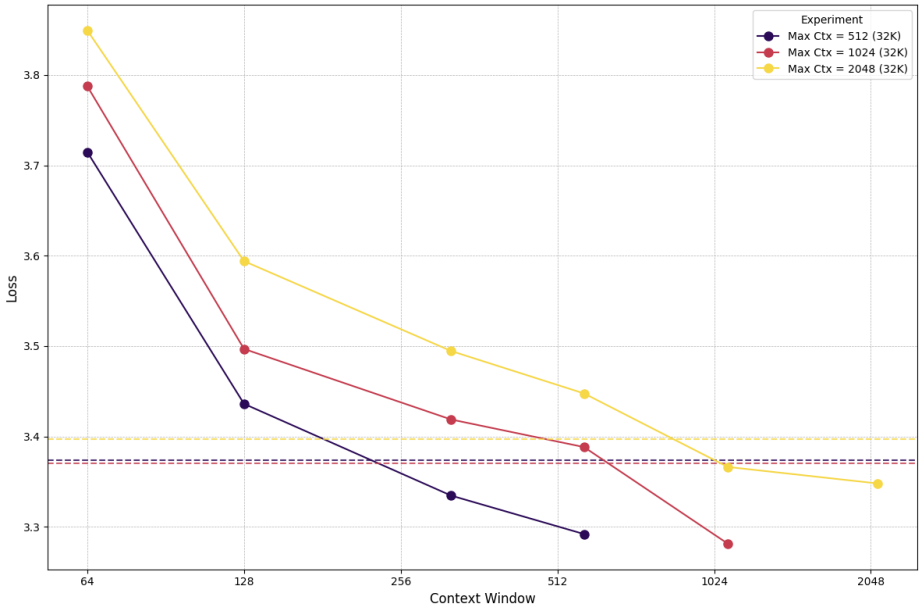

Figure 5: Context-length sweep for the 5M-parameter model (batch size 32k tokens). In contrast to the 1M case, the 5M model handles a 1024-token training context well (slightly better than 512), but starts to lose performance at 2048. This links model size to the largest maximum context length that can be used while keeping performance optimal.

B Ablation Studies for Knowledge Distillation Scheduling

This section provides supplementary ablations that help interpret why knowledge distillation (KD) provides most of its benefit early, and why learning-rate scheduling becomes the dominant bottleneck later in training. These observations motivate the paper's focus on *bootstrapped* KD as a way to skip expensive early optimization steps on the larger model (Section 4).

Learning-Rate Sensitivity. Across model sizes, we found that learning rate strongly controls late-stage progress: improvements beyond an early-loss plateau typically required the learning rate to decay further. This interacts with KD in two ways. First, KD can tolerate larger learning rates early, enabling rapid initial convergence. Second, to preserve that advantage, the schedule must still reach sufficiently low learning rates in time; otherwise, the run becomes *learning-rate limited* and the distilled and classical trajectories merge (as illustrated in figure 6).
Figure 7 reinforces this point: once KD ends, runs at similar learning rates achieve near-identical validation loss regardless of training step, confirming that learning rate largely determines achievable loss under comparable conditions. Building on this insight, figure 8 shows that a KD-aware schedule—using a higher learning rate during the initial KD phase followed by a sharp drop when KD stops—can translate early KD gains into fewer total steps to reach a target loss.

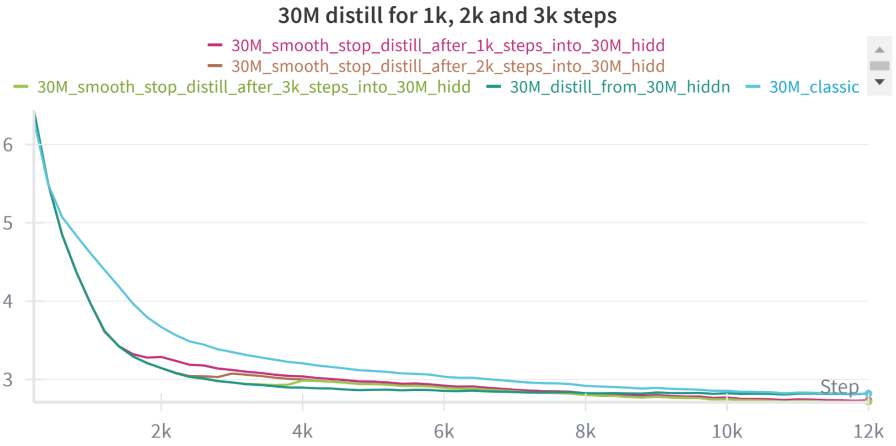

Figure 6: Distillation-duration ablation (30M student): applying KD for only the first 1k/2k/3k steps captures most of the gain. After KD stops, runs follow similar trajectories, indicating that KD primarily shifts the early phase while later progress is dominated by the learning-rate schedule.

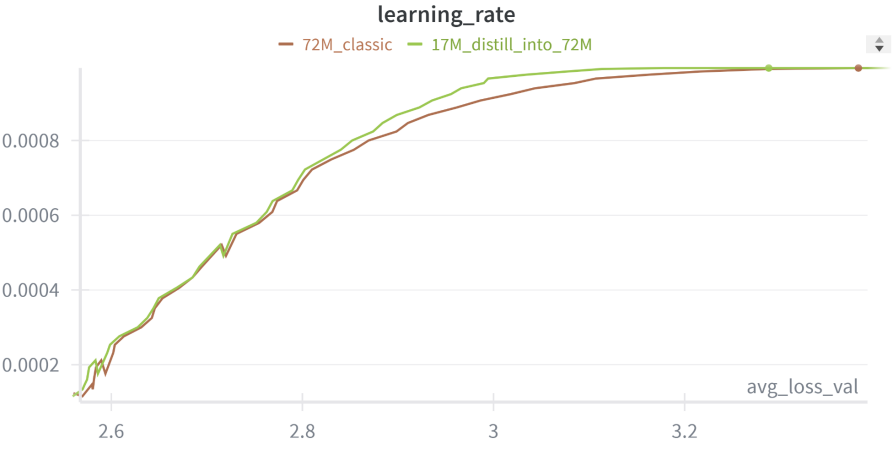

Figure 7: Learning rate vs validation loss for 17M→70M bootstrapped distillation vs classical training. Once KD ends, runs at similar learning rates exhibit near-identical loss regardless of training step—suggesting that, given comparable training conditions, learning rate largely determines achievable loss.

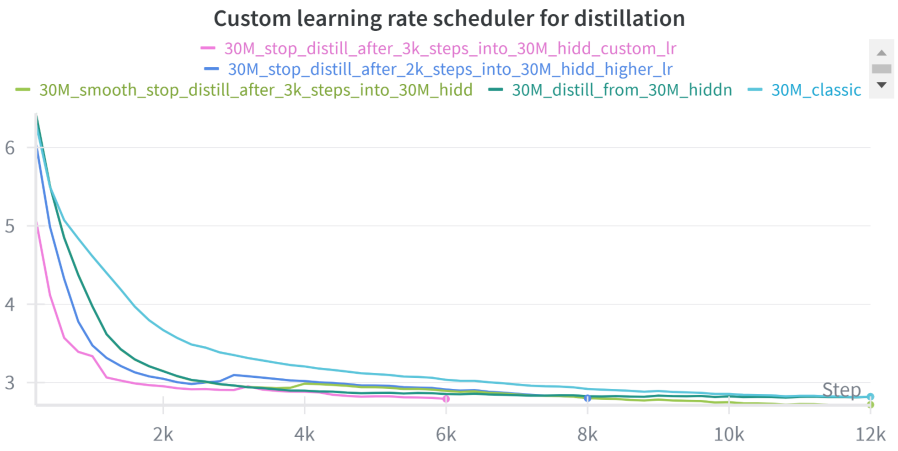

Figure 8: KD-aware learning-rate scheduling: using a higher learning rate during the initial KD phase, followed by a sharp drop when KD stops, can convert early KD acceleration into fewer total steps to reach the same loss.